

Función de cámara: qué falta y cómo se va a construir

Plan de implementación de la grabación sincronizada con las lecturas RFID — estado: pendiente, sin implementar. Documento generado el 2026-09-01 — actualizado el 2026-09-11 con una segunda forma posible de hacerlo (ver sección 9, al final).

1. Qué pide el proyecto

La carpeta técnica original de CowControl (el documento presentado ante el colegio) exige una función de cámara vinculada al conteo. No alcanza con una foto suelta — pide específicamente **video**, con cada lectura de tag ubicada en el momento exacto en que ocurrió dentro de la grabación:

"...los tiempos de grabación sincronizados y especificados cuando el animal en particular se acerca al sensor. Ejemplo: el ID número 2387 pasó en el minuto 8:33. La grabación queda guardada en la app y puede ser revisada cuando sea deseado."

El plan original (de antes de que existiera la app Flutter) ya proponía usar la webcam de una notebook **Juana Manso** como cámara. Hoy no hay ningún módulo de cámara comprado ni planeado para la Raspberry Pi — se retoma esa idea original.

2. Estado actual

100% PENDIENTE

No existe ningún código, tabla, ni script relacionado con esto todavía. Este documento describe el plan a seguir, no algo ya construido.

3. Cómo se eligió la arquitectura

Se evaluaron 3 propuestas de arquitectura distintas e independientes para resolver el mismo problema (grabar con la Juana Manso y recortar automáticamente un clip corto de 2-3 segundos por cada tag leído, en vez de guardar el video completo, que sería muy pesado):

Propuesta	Idea central	Resultado
Edge / todo local	La notebook graba y corta los clips ella misma con ffmpeg. Solo suben los clips livianos a Supabase.	✓ Elegida
n8n como orquestador	La idea original de Santiago: n8n corriendo en la notebook, controlando todo el proceso con workflows visuales.	Descartada
Serverless (sin servidor propio)	El video sube completo a Supabase, un job de GitHub Actions lo baja, lo corta y sube los clips.	Descartada

Por qué se descartó n8n

Depende de que alguien se acuerde de prenderlo a mano antes de cada conteo, su función para ejecutar comandos (necesaria para correr ffmpeg) viene desactivada por defecto por seguridad, y su versión gratuita en la nube ya no existe de forma permanente. No resuelve nada que un script simple no resuelva más fácil, y es una herramienta nueva que nadie del equipo conoce.

Por qué gana la opción "todo local"

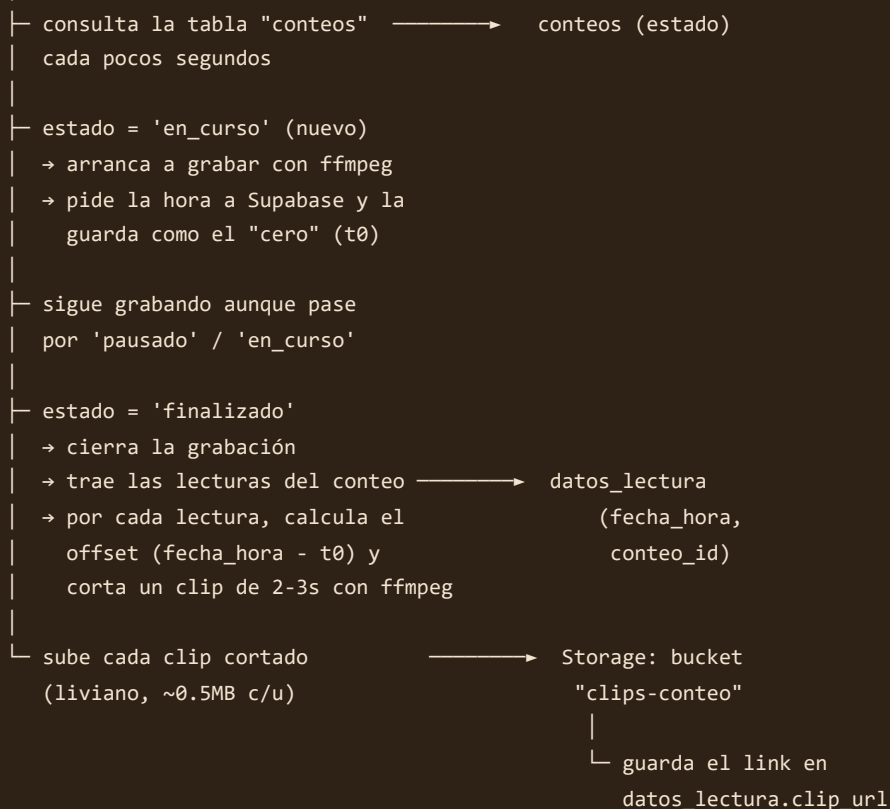
Es la única de las tres donde el video pesado nunca sale de la notebook — ni sube ni baja de ningún lado. Usa el mismo tipo de trabajo (Python + pedidos HTTP a Supabase) que ya se usa en `rfid_tick.py` para la Raspberry Pi, así que no hace falta aprender ninguna herramienta nueva. Es también la opción más barata en el uso real de la cuota gratuita de Supabase.

4. Arquitectura elegida

Notebook Juana Manso

Supabase

script Python en loop



5. Qué hay que agregar o cambiar

5.1 — Base de datos (Supabase)

Dos cambios chicos, se pueden correr directo en el SQL Editor:

```
-- bucket nuevo para los clips (se crea desde el panel de Storage, no por SQL)
-- nombre: clips-conteo

-- columna nueva para guardar el link del clip de cada lectura
alter table datos_lectura add column clip_url text;
```

5.2 — Script nuevo en la notebook (Python)

Un script que corre en la Juana Manso mientras dura el conteo. Reutiliza el mismo patrón de `rfid_tick.py` (pedidos HTTP simples a Supabase con la `service_role key`), sumando:

- Un loop que consulta el estado del conteo activo cada pocos segundos.
- Arranque/cierre de `ffmpeg` como subprocesso, grabando la webcam.
- Al finalizar: por cada lectura, cálculo del punto exacto dentro del video y corte del clip con `ffmpeg`.
- Subida de cada clip a Supabase Storage y actualización de `clip_url` en la lectura correspondiente.

Ejemplo del comando de grabación (Windows, webcam vía DirectShow):

```
ffmpeg -f dshow -video_size 640x480 -framerate 15 -i video="Integrated Camera" ^
-c:v libx264 -preset ultrafast -pix_fmt yuv420p grabacion.mp4
```

Ejemplo del corte de un clip puntual:

```
ffmpeg -ss {inicio_segundos} -i grabacion.mp4 -t 3 ^
-c:v libx264 -preset ultrafast -c:a aac -y clip.mp4
```

5.3 — App Flutter

- Agregar el campo `clipUrl` al modelo `Lectura`.
- Sumar el paquete `video_player` (oficial, gratis) a `pubspec.yaml`.
- Un botón/pantalla nueva para reproducir el clip — mismo patrón visual que los botones de Fertilidad/Vacunas/Observaciones que ya existen en la ficha del animal.

6. Bloqueante a resolver antes

BLOQUEA TODO LO DEMÁS

`rfid_tick.py` (el script que ya corre en la Raspberry Pi) todavía no manda `conteo_id` en cada lectura que inserta en `datos_lectura` — solo manda `rfid_uid`, `campo_id` y `fecha_hora`. Sin `conteo_id`, no hay forma de saber qué lecturas pertenecen a qué conteo/video. Hay que confirmar con el compañero a cargo del hardware si ya lo actualizó o sigue pendiente, y coordinar el cambio si falta.

7. Pasos a seguir, en orden

- 1 **Confirmar el bloqueante** — si `conteo_id` ya se está mandando desde la Raspberry Pi o sigue pendiente.

2 Prueba aislada de hardware — correr a mano los dos comandos de ffmpeg (grabar la webcam, cortar un clip) directo en la Juana Manso real, sin nada de Supabase todavía, para confirmar que la cámara coopera.

3 Cambios chicos en Supabase — crear el bucket `clips-conteo` y agregar la columna `clip_url`.

4 Script de arranque/parada en la notebook — el loop que vigila `conteos` y prende/apaga la grabación.

5 Lógica de corte y subida — recorrer las lecturas del conteo, cortar un clip por cada una, subir a Supabase.

6 Botón/pantalla en Flutter para reproducir el clip.

7 Prueba de punta a punta con hardware real, coordinando con el compañero a cargo de la Raspberry Pi — con margen antes de la presentación.

8. Riesgos conocidos (aceptados para un prototipo de secundaria)

- **No probado en el hardware real todavía** — si la webcam de esa notebook específica no coopera con ffmpeg, puede hacer falta ajustar resolución o cuadros por segundo.
- **La notebook tiene que estar prendida, sin suspenderse, y bien apuntada** durante todo el conteo — un detalle de armado del stand, no de software.
- **Segunda credencial (`service_role key`) en circulación** — mismo modelo de confianza ya aceptado para la Raspberry Pi, simplemente duplicado. Razonable para un prototipo, no para un producto real.

9. Dos formas posibles de hacerlo — sin decidir todavía entre ellas (actualización 2026-09-11)

El equipo (Germaioni) planteó una segunda forma de armar la función de cámara, distinta a la de la sección 4 de arriba. **Ninguna de las dos está descartada** — esta sección explica las dos, y lo que hay que entender de Supabase Storage para poder elegir con criterio. No se recomienda ninguna acá a propósito.

9.1 — Camino A: clips cortos (el de la sección 4)

Ya explicado arriba en detalle. En resumen: la notebook graba todo el conteo, y al finalizar corta con ffmpeg un clip de 2-3 segundos por cada lectura. Se suben muchos archivos chicos a Supabase Storage, y cada lectura queda con su propio `clip_url` apuntando a su clip individual.

9.2 — Camino B: un video completo, con texto superpuesto y salto directo a un segundo (propuesta nueva del equipo)

La idea, tal como la planteó Germaioni: no se corta nada. Se sigue un solo video de punta a punta, y:

- Por cada lectura, se le superpone un texto sobre el video (a la izquierda de la imagen) con el UID del animal, que aparece solo durante el momento exacto en que ese animal pasó — un cartel que entra y sale en el instante justo. Técnicamente esto se logra con el filtro `drawtext` de ffmpeg, activado con una condición de tiempo (`enable='between(t,inicio,fin)'`) en vez de cortar el archivo.
- Por cada lectura, se guarda el **segundo exacto** en el que pasó ese animal — un número, no un archivo nuevo.
- En la app, en vez de un botón que abre un clip aparte, hay un link que abre **el mismo video de siempre** pero arranca reproduciendo directo desde ese segundo — el mismo concepto que un timestamp de YouTube: tocás el comentario con el minuto marcado, y el video salta ahí solo, sin descargar nada nuevo.
- Técnicamente, esto se arma con `VideoPlayerController.seekTo(Duration(seconds: segundo))` en el paquete `video_player` de Flutter, después de cargar el video una sola vez.

9.3 — Supabase Storage a fondo (para entender el compromiso real de cada camino)

Qué es: un servicio de almacenamiento de archivos (fotos, videos, PDFs, cualquier binario) integrado a Supabase, organizado en "buckets" — carpetas de primer nivel. Cada archivo que se sube ahí queda con una URL propia para descargarlo o mostrarlo desde la app, igual que ya se usa para el APK en GitHub Releases, pero del lado de Supabase.

Cómo limita el plan gratis (verificado por búsqueda real al momento de decidir la sección 4, no son números de memoria):

Límite	Qué significa
1 GB de almacenamiento total	La suma de TODOS los archivos que haya subidos en la cuenta, acumulado, no se resetea nunca solo.
50 MB por archivo individual	Sin importar cuánto espacio libre quede del 1GB total, un solo archivo no puede pesar más que esto en el plan gratis. Es un tope por archivo, no por cuenta.
Egress (tráfico de descarga/reproducción)	5 GB/mes sin caché + otros 5 GB con caché — se gasta cada vez que alguien mira o descarga algo desde la app.

Qué implica esto para cada camino, en números concretos:

- **Camino A (clips cortos):** cada clip pesa aproximadamente 0.3-0.8 MB. Entran más de 1000 clips en el 1GB total, y ninguno se acerca ni de lejos al límite de 50MB por archivo. Margen muy amplio.
- **Camino B (video completo):** un video de varios minutos, incluso a resolución baja (640x480), puede pesar fácilmente varias decenas o cientos de MB según cuánto dure el conteo. **Acá es donde puede chocar directo con el límite de 50MB por archivo** — si el conteo dura varios minutos, es probable que el video completo directamente no se pueda subir a Supabase Storage en el plan gratis, sin importar cuánto espacio total quede disponible. Además, cada vez que alguien reproduce ese video completo desde la app (aunque solo quiera ver 3 segundos, saltando a un punto puntual) puede llegar a descargar bastante más datos que con un clip cortito, dependiendo de cómo maneje el reproductor el streaming del archivo.

Esto no es un "no se puede" del Camino B — es un "hay que resolverlo" si se lo elige. Algunas opciones que existen para eso (mencionadas acá solo para que se sepa que hay caminos, sin recomendar ninguna en particular): grabar a resolución/bitrate bajo para que el archivo final pese menos; usar otro servicio de almacenamiento de video con otros límites en vez de Supabase Storage para este archivo puntual; o encontrar alguna forma de acortar/comprimir el video antes de subirlo.

9.4 — Comparación lado a lado

	Camino A — clips cortos	Camino B — video completo + salto
Qué se sube a Storage	Muchos archivos chicos (uno por lectura)	Un solo archivo grande (uno por conteo)
Riesgo de chocar con el límite de 50MB	Prácticamente nulo	Real, depende de la duración del conteo
Qué ve el productor	Un clip cortito, aislado, de ese animal puntual	El video completo del conteo, con la opción de saltar al momento de cada animal
Contexto alrededor de cada lectura	No — el clip empieza y termina justo en ese animal	Sí — se puede seguir viendo lo que pasó antes/después dentro del mismo video
Qué hay que construir	Lógica de corte con ffmpeg por cada lectura + subida de varios archivos	Lógica de superponer texto en el momento justo + guardar un número (segundo) por lectura, un solo archivo para subir

CowControl — documento generado a partir del análisis de arquitectura del 2026-09-01 (documentacion.md/cosas_pendientes.md, ítem 26), actualizado el 2026-09-11 con la propuesta del equipo sobre el Camino B. Nada de esto está implementado todavía — ninguno de los dos caminos fue decidido como definitivo.